

ZENITH

Cloud Resource Optimization Platform

Comprehensive Project Documentation

Version 1.0 — May 2026

Developer: A. Prithviraj

Project Name	Zenith (CloudResourceOptimizationPlatform)
Type	Full-Stack Multi-Cloud Storage Management Platform
Frontend	React 19 + Vite 7 (JavaScript)
Backend	FastAPI (Python) + Celery + MongoDB Atlas
Cloud Providers	AWS S3, Google Cloud Storage, Azure Blob Storage
Security	JWT Auth + TOTP 2FA + AES-256 Encryption

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Project Architecture
4. Technology Stack
5. Core Features & Modules
6. Intelligent Storage Placement Algorithm
7. Security Architecture
8. Async Processing & Background Jobs
9. API Reference
10. Database Schema
11. DevOps & Infrastructure
12. Future Roadmap

1. Executive Summary

Zenith is a full-stack, multi-cloud storage management platform designed to intelligently optimize file placement across AWS S3, Google Cloud Storage, and Azure Blob Storage. The platform leverages a scoring-based ML algorithm to automatically recommend the most cost-effective cloud provider and storage tier for each uploaded file.

Key differentiators include:

- Intelligent file placement using a multi-factor scoring algorithm
- Multi-cloud support across AWS, GCP, and Azure with unified API
- Two-Factor Authentication (TOTP) protected secure file vault
- Automatic sensitive data scanning with regex-based detection
- AES-256 server-side encryption with cross-region replication
- Async background processing via Celery with RabbitMQ broker
- Real-time WebSocket notifications for background job completion
- Automated nightly storage tier optimization (Hot→Warm→Cold lifecycle)

2. Problem Statement & Motivation

Organizations today face significant challenges in managing cloud storage costs across multiple providers. Without intelligent placement strategies, files are often stored in expensive hot-tier storage regardless of access patterns, leading to unnecessary costs.

2.1 Challenges Addressed

- **Cost Inefficiency:** Files stored in hot tiers when rarely accessed waste 60-80% of storage budget
- **Multi-Cloud Complexity:** Each CSP has different storage classes, pricing models, and APIs
- **Security Gaps:** Sensitive files often lack proper encryption and access controls
- **Manual Management:** Storage tier transitions require manual intervention and monitoring
- **No Lifecycle Automation:** Files sit in the same tier indefinitely without optimization

2.2 Solution Approach

Zenith solves these problems through a three-pronged approach: (1) an intelligent scoring algorithm that analyzes file metadata to recommend optimal placement, (2) automated nightly lifecycle management that demotes idle files and promotes active ones, and (3) a secure file vault with 2FA protection, automatic sensitive data scanning, and AES-256 encryption.

3. Project Architecture

Zenith follows a monorepo architecture with cleanly separated frontend and backend sub-projects communicating via RESTful APIs and WebSocket connections.

3.1 High-Level Architecture

Layer	Technology	Purpose
Frontend	React 19 + Vite 7	SPA with dashboard, storage management, security vault
API Gateway	FastAPI + Uvicorn	RESTful API with 7 router modules
Auth Layer	JWT + TOTP 2FA	Stateless authentication with optional 2FA enforcement
Database	MongoDB Atlas	Document store for users, files, secure_files
Task Queue	Celery + CloudAMQP	Async file processing, encryption, nightly optimization
Cloud Storage	AWS S3 / GCP / Azure	Multi-cloud file storage with tier management
Real-time	FastAPI WebSocket	Push notifications for background job completion

3.2 Monorepo Structure

The project is organized as a monorepo with the following top-level structure:

Directory	Purpose
frontend/	React 19 SPA — pages, components, styles, context
backend/	FastAPI application — auth, storage, security, websockets
backend/app/auth/	JWT authentication, password hashing, 2FA utilities
backend/app/storage/	Multi-cloud upload, download, delete, optimizer, tiering
backend/app/security/	Secure file vault, 2FA routes, encryption pipeline
backend/app/websockets/	Real-time notification system via WebSocket
backend/app/database/	MongoDB Atlas client singleton
devops/	Terraform IaC, Docker configs, CI/CD pipelines
docs/	Project documentation

4. Technology Stack

4.1 Frontend Stack

Technology	Version	Purpose
React	19.1.1	UI framework — functional components only
Vite	7.1.0	Build tool and dev server
React Router DOM	6.30.1	Client-side routing (createBrowserRouter)
Axios	1.11.0	HTTP client for API communication
React Toastify	11.0.5	Toast notification system
ESLint	9.32.0	Code linting and quality

4.2 Backend Stack

Technology	Purpose
FastAPI	ASGI web framework — title: "Zenith API"
Uvicorn	ASGI server for FastAPI
MongoDB Atlas (pymongo)	Cloud document database
python-jose (JWT)	Token-based authentication (HS256)
passlib + bcrypt	Password hashing
pyotp + qrcode	TOTP 2FA generation and verification
Celery + CloudAMQP	Distributed task queue with RabbitMQ broker
boto3	AWS SDK — S3 operations
google-cloud-storage	GCP SDK — Cloud Storage operations
azure-storage-blob	Azure SDK — Blob Storage operations
Pydantic + pydantic-settings	Data validation and config management

5. Core Features & Modules

5.1 Authentication System

The authentication system uses JWT tokens with bcrypt password hashing. Users register with username, email, and password. On login, a JWT token is issued with the username as the subject claim. The token is stored in browser localStorage and sent with every API request via the Authorization header.

5.2 Multi-Cloud Storage Management

The storage module supports uploading, downloading, listing, and deleting files across three cloud providers. Each file is associated with metadata stored in MongoDB, including the CSP, storage class, access frequency, and timestamps. The system supports pre-signed URL generation for secure downloads.

5.3 Secure File Vault (2FA Protected)

The security module provides a separate, 2FA-protected file vault. Files uploaded here are stored in a dedicated S3 bucket and processed asynchronously by Celery workers. Processing includes sensitive data scanning (credit card numbers, API keys) and optional AES-256 server-side encryption with cross-region replication.

5.4 Dashboard

The dashboard provides an overview of the user's cloud resources with stat cards showing monthly costs, virtual machines, storage usage, and active alerts. The layout includes a responsive sidebar with navigation to all major sections.

5.5 Real-time Notifications

WebSocket connections provide real-time push notifications. When a Celery background task completes (e.g., file encryption), it calls an internal HTTP endpoint which bridges the notification to the user's active WebSocket connection. The frontend automatically refreshes data upon receiving these notifications.

6. Intelligent Storage Placement Algorithm

The optimizer module implements a scoring-based algorithm that recommends the optimal cloud provider and storage tier for each file upload.

6.1 Scoring Factors

Factor	Criteria	Score Impact
User Priority	Cost-focused	+5 (favors cold)
User Priority	Performance-focused	-5 (favors hot)
User Intent	Archival	+15
User Intent	Infrequent access	+10
Filename Keywords	backup, archive, export, log	+5
Date Pattern	YYYY-MM-DD in filename	+3
File Type	Archive (.zip, .tar, .gz)	+5
File Type	Media (.jpg, .mp4)	-5
File Size	> 1 GB	+5
File Size	> 100 MB	+3

6.2 Tier Classification

Score > 12: Cold tier (archival) — Lowest cost, highest retrieval latency
Score 6-12: Warm tier (infrequent access) — Moderate cost and latency
Score ≤ 5: Hot tier (frequent access) — Highest cost, instant retrieval

6.3 CSP Selection

After determining the tier, the algorithm compares pricing across all three CSPs for that tier, factoring in retrieval penalties and minimum duration costs. The cheapest effective option is recommended, while all alternatives are provided for user override.

Tier	AWS	GCP	Azure
Hot	S3 Standard (\$0.023/GB)	Standard (\$0.020/GB)	Hot Blob (\$0.018/GB)
Warm	S3 Standard-IA (\$0.0125/GB)	Nearline (\$0.010/GB)	Cool Blob (\$0.010/GB)
Cold	Glacier Flexible (\$0.004/GB)	Archive (\$0.0012/GB)	Archive (\$0.00099/GB)

7. Security Architecture

7.1 Authentication Flow

1. User registers → password hashed with bcrypt → stored in MongoDB
2. User logs in → JWT created with sub:username → stored in localStorage
3. Protected routes: get_current_user() dependency decodes JWT
4. If 2FA enabled: login resets two_fa_verified → must re-verify
5. require_2fa dependency enforces verification on security endpoints

7.2 Two-Factor Authentication

TOTP-based 2FA using pyotp with QR code generation. Issuer: "ZenithApp". The flow includes: enable (generate secret + QR), finalize (first code verification), verify (session verification), and disable.

7.3 Encryption & Replication

Sensitive files are encrypted using AES-256 server-side encryption (SSE) and replicated to a secondary S3 bucket in a different AWS region for disaster recovery.

Bucket	Purpose	Region
zeneith-storage-bucket	Standard multi-cloud storage	Default
zenith-secure-files	2FA-protected secure uploads	Default
zeneith-secure-files-secondary2	Encrypted file replication	us-east-1

8. Async Processing & Background Jobs

8.1 Celery Task: `process_secure_file`

Triggered when a file is uploaded to the secure vault. The task:

1. Downloads the file from S3 for scanning
2. Scans text-based files for credit card patterns and secret keywords
3. If sensitive or manually flagged: encrypts with AES-256 SSE
4. Replicates encrypted file to backup bucket
5. Updates MongoDB with `is_sensitive` and `is_encrypted` flags
6. Sends WebSocket notification to the user

8.2 Celery Scheduled Task: `run_storage_optimization`

Runs nightly at 00:00 UTC via Celery Beat. Performs two operations:

Demotion Logic:

- Hot → Warm: Files idle for 30+ days
- Warm → Cold: Files idle for 90+ days

Promotion Logic:

- Cold → Warm or Warm → Hot: Files accessed >2 times in the last 5 days
- Promotions are one-level-up only (Cold→Warm, not Cold→Hot)

9. API Reference

Method	Endpoint	Auth	Purpose
GET	/	No	Welcome message
POST	/api/auth/register	No	Register new user
POST	/api/auth/token	No	Login — get JWT token
GET	/api/users/me	JWT	Get current user profile
GET	/api/dashboard/stats	JWT	Dashboard statistics
POST	/api/storage/analyze	JWT	Analyze file for optimal placement
POST	/api/storage/upload	JWT	Upload file to CSP
GET	/api/storage/files	JWT	List user files
GET	/api/storage/download/{fn}	JWT	Pre-signed download URL
DELETE	/api/storage/delete/{fn}	JWT	Delete file from CSP
POST	/api/security/upload-secure	2FA	Upload to secure vault
GET	/api/security/list-secure	2FA	List secure files
GET	/api/security/download/{fn}	2FA	Secure file download URL
DELETE	/api/security/delete/{fn}	2FA	Delete from both buckets
GET	/api/2fa/status-2fa	JWT	Check 2FA status
POST	/api/2fa/enable-2fa	JWT	Generate QR code
POST	/api/2fa/finalize-2fa	JWT	Enable 2FA (first verify)
POST	/api/2fa/verify-2fa	JWT	Session 2FA verification
POST	/api/2fa/disable-2fa	JWT	Disable 2FA
WS	/ws/status?token=...	JWT	Real-time notifications
POST	/ws/notify/{user_id}	None	Internal: Celery→WS bridge

10. Database Schema

MongoDB Atlas database: **CloudResourceOptimizationDB**

10.1 Users Collection

Field	Type	Description
username	string	Unique identifier (3-50 chars)
email	string (EmailStr)	User email address
hashed_password	string	bcrypt hashed password
two_fa_secret	string (optional)	TOTP secret key
two_fa_enabled	boolean	Whether 2FA is active
two_fa_verified	boolean	Session verification status

10.2 Files / Secure_Files Collections

Field	Type	Description
filename	string	Original file name
s3_key	string	Object key (username/filename)
owner_username	string	File owner
size_bytes	integer	File size in bytes
upload_date	datetime	Upload timestamp
csp	string	Cloud provider (AWS/GCP/Azure)
storage_class	string	Current storage tier
last_accessed_at	datetime	Last download timestamp
access_frequency_score	integer	Access counter for tiering
is_sensitive	boolean	Sensitive data detected
is_encrypted	boolean	AES-256 encryption applied

11. DevOps & Infrastructure

11.1 Running the Application

The application requires 4 terminal sessions:

Terminal	Command	URL/Purpose
1: Backend	<code>uvicorn app.main:app --reload</code>	<code>http://localhost:8000</code>
2: Frontend	<code>npm run dev</code>	<code>http://localhost:5173</code>
3: Celery Worker	<code>celery -A app.celery_worker worker</code>	Processes async tasks
4: Celery Beat	<code>celery -A app.celery_worker beat</code>	Nightly scheduler

11.2 Infrastructure

Terraform configurations exist for AWS resources (S3, IAM, DynamoDB, Lambda). Docker configurations include separate Dockerfiles for frontend and backend with a `docker-compose.yml`. CI/CD pipeline configurations are available for both GitHub Actions and GitLab CI.

12. Future Roadmap

Priority	Feature	Description
High	Cost Analysis Page	AWS Cost Explorer integration with visualizations
High	Compute Management	EC2 instance management and auto-scaling
High	Real Dashboard Data	Connect dashboard stats to actual cloud metrics
Medium	ML Cost Predictions	Train models for cost forecasting
Medium	Audit Logging	Security event tracking and compliance
Medium	KMS Encryption	AWS KMS managed encryption keys
Low	Docker Deployment	Production-ready containerization
Low	CI/CD Pipeline	Automated testing and deployment

— End of Document —